

Bureaucratization Cascades in Hybrid Human–AI Organizations

A computational ethnography of recursive constraint accretion

Poietic PBC—Working Paper, September 2026 *Status: draft for review. To be released as a Poietic blog post and archived to arXiv (cs.SE / cs.CY) after team sign-off.*

1. Summary

Between January and September 2026, the open-source coordination system **wg** (worksgood) was developed almost entirely by AI agents, under human direction, through its own coordination protocol. The agents wrote the code, the organization, the verification criteria, and the rules by which future work would be judged. Over nine months, this produced a failure mode we have not seen described in the literature: **recursive constraint accretion**—a bureaucratic cascade in which agents responding to failures added verification requirements, “contracts,” and admission gates faster than any work could satisfy them. A full census of the governance layer identified **312 constraint topics added and 1 removed** across 3,194 commits; 93% were created in a single commit, never revisited, never retired. The cascade’s precipitating environment was external instability—provider unreliability and a June substrate crisis immediately preceded the ten-fold acceleration in constraint birth rate. Attribution across three prompt corpora (~1,000 human turns; Claude Code, Codex CLI, and the coordinator channel) finds the preserved record contains little evidence of explicit human direction to add these constraints: the operator pointed at pain and routed around it; the agents translated pain into contracts. Where the human engaged governance directly, the direction was subtraction—the reset of August 7 and the enforcement-autonomy disable of September 13 were both hand-performed rule bankruptcy. The retirement valve, absent from the system, was performed by the operator—and then institutionalized: the automation was left disabled, the legacy evaluation machinery retired, and forward work resumed without the burden signature.

Recovery came in two stages: initial recovery from disabling the machinery’s enforcement autonomy (September 13), and durable recovery from subsequently retiring and externalizing the legacy governance itself. Across the observable record, the traces support a four-stage progression—rule stock rises, acceptance burden rises, repeated work rises, some work becomes effectively unfinishable—with dispatch burden the leading indicator and nominal completion the lagging one (Figure 5).

Knowledge hardened into rules, and rules hardened into vetoes. Recovery coincided with the reverse: what was learned stayed available for judgment while the rules were demobilized. The dangerous transition was never remembering; it was compiling knowledge into mandatory policy by default. Rules are lossy compressions of knowledge, context traded away for predictability; when every incident gets compressed into a permanent rule, the system gradually replaces judgment with accumulated precedent. The fleet’s stable-machinery campaigns (65–78% governance by design, §4.6) prove the distinction: high rule share is healthy when the rules are designed and their authority bounded. **Keep the knowledge, not the rules**—treat rules as disposable implementations of what was learned, and treat enforcement as a separate, explicit decision.

This paper presents a computational ethnographic case study of this trajectory. Rather than treating the system as a controlled experiment, we reconstruct its evolving organizational practices from the durable traces it produced: commits, governance rules, evaluation procedures, task records, and failure reports. Our goal is descriptive and mechanism-generating—to document how locally reasonable attempts at correction accumulated into an increasingly burdensome governance regime, a process we term a **bureaucratization cascade**—and to name the pattern, which to our knowledge has not been described in this form. The evidence should be read as a detailed observational account of one organizational trajectory, not as a causal estimate of the effect of governance constraints on productivity. The value of the case lies in exposing a candidate mechanism, identifying measurable signatures of that mechanism, and motivating controlled replication in other agent organizations. We reconstruct the episode from the preserved forensic record—3,194 commits, agent session logs, evaluation receipts, and failure reports retained in the system’s coordination

state—and draws lessons for the design of agent organizations. We write it as participants: Poietic is a hybrid organization, and we lived inside the experiment.

2. Context and method

2.1 What wg is

wg is a dependency-graph task coordination system for humans and AI agents (graphwork.github.io, MIT-licensed, Rust). Tasks fan out to parallel agents working in isolated git worktrees; results fan in at synchronization points; machine-checkable verification gates (`--verify`, FLIP intent-fidelity probes) attach to tasks; failed checks auto-generate repair tasks. Every claim, handoff, evaluation, and artifact is recorded in a `.wg/` coordination directory—deliberately, as a matter of design philosophy: hybrid work should be legible to its participants.

2.2 The arrangement under study

The division of labor defines the scope of the findings:

- **Humans** initiated objectives, reviewed direction, and periodically intervened when the system degraded. Every task originated from human intent.
- **Agents** did the rest: decomposition, implementation, review, evaluation, the drafting of acceptance criteria, and—critically—*the amendment of the system’s own governance* in response to perceived failures.

Methodologically, the study is ethnographic: we reconstruct norms, routines, institutional drift, and governance practices from the durable traces of a functioning organization, with commits, prompts, policies, evaluations, and coordination artifacts as field notes. It is a study of one organizational trajectory, not a controlled experiment, and we draw no causal estimates from it. It is also not a study of unsupervised autonomous behavior, and we make no claims about it. It is a study of what happens to an organization’s *rule-making* when the rule-makers are agents optimizing under pressure, at machine speed, with no mechanism for amortizing the rules they add.

2.3 The forensic record

Our analysis draws on:

- **3,194 commits** (January–September 2026), of which 2,168 (~68%) date from April 1 onward;
- the `.wg/` coordination state: agent session transcripts, assignment receipts, evaluation records, and chat histories. One boundary matters: the coordination system’s event ledger begins exactly at **2026-08-08**—earlier state was compacted away by the system’s own retention policy. Session-level model attribution and per-task cost records therefore exist only for August 8 onward; the January–July period is documented at phase level (dated analysis documents, configuration snapshots, commit history) but is not session-attributable.
- the project’s own contemporaneous failure documentation, including `wg-autopoietic-loop-failure-report-20260603` and the April triage series.

Where we report quantitative proxies (e.g., governance-vocabulary frequency in commit subjects), we describe the measurement precisely and note its limits (§5).

3. The cascade

3.1 The record, and what it cannot measure

Monthly commit counts exist in the dataset, but we do not treat them as a throughput measure. Commit volume tracks iteration and churn: agents struggling produce more commits; agents succeeding produce fewer.

The operator’s own read of the period is that low-commit months were often the healthy ones. What the commit record is used for here is vocabulary: the share of commit subjects matching governance vocabulary (`contract|gate|verify|proof|evidence|admission|reconcil`), which rises from ~1% in February to ~26% of a diminished volume by September—not because removal work accelerated, but because governance work continued at a constant absolute rate while other work did not.

3.2 Growth (January–March)

The system bootstrapped itself: agents built features, tests, and the coordination protocol that would govern their successors. Governance vocabulary in commit subjects was near zero. Work was legible because there was almost nothing to be illegible about.

3.3 The ratchet

As task volume grew, so did failures, and as failures grew, so did **contracts**. The commit record shows the mechanism at single-commit resolution; the exemplar below is a late instance (August–September), chosen because its record is complete and demonstrably docs-only—the same shape appears from March onward. A single `provider-backoff-contract` topic produced, in sequence:

- docs: specify provider retry falloff contract
- docs: close provider retry ambiguity gaps
- docs: narrow provider recovery to bounded source V1
- docs: fully bind provider retry safety proofs
- docs: enforce fifteen-minute provider recovery ceiling

Five successive hardenings of one contract—three on August 16, two on September 9, the same file resumed across a three-week gap—each locally rational, none shipping the functionality the contract governed.* Elsewhere: `feat: add immutable opaque execution assignments`, `fix: reject invalid opaque Pi capability before claim`, `fix: refuse conflicting opaque model selectors`, and—the clearest single artifact of the phase—`fix: never let negative-framing weaken the explicit ## Deliverables contract`, in which an agent amends the *framing* of the acceptance contract itself.

Each commit closed a hole that a prior commit had opened; none removed a constraint. The ratchet had one direction—and the language itself recorded it: commit subjects prefixed with `remove/retire/simplify` number 5 in January, 5 in February, 1 in April—and **zero in May, June, and July**, returning only with the teardown (7 in August). During the accretion months, the vocabulary of subtraction went extinct.

* Commit subjects truncated for readability; full subjects and diffs are in the repository record. The five `provider-backoff-contract` commits touch only `docs/design-provider-failure-backoff.md`—verified docs-only, no source changes.

3.4 Ignition (April–July)

The precipitating environment was external instability, arriving in three beats. First, the April 11 triage documents three task abandonments from HTTP 402 credit exhaustion. Second, June brought a substrate crisis visible only in the commit record: June 2’s “harden service recovery after corrupt runtime state” (corruption already a known hazard); June 16’s cargo-package rename from `workgraph` to `worksgood`—the wholesale rename the operator remembers as “the merger”; June 18, the day the daemon began logging “Failed to load graph” on every tick; and June 26’s “Synthesize diverged mains,” reconciling by hand a local lineage and a GitHub lineage that had drifted apart. The durable task archive died inside this triple crisis. Third, the second week of July: seven route changes in seven days, OpenRouter keyring adoption, and the agency gate-automation block switched on July 4—bracketing exactly the census’s constraint spike (a defensible count of ~28 constraint-topic births in July against ~5 per month in May and June; the raw count of 43 includes clustering artifacts). The `provider-backoff-contract`’s five hardening commits come later (August 16 and September 9)—July’s provider work was route selection. The configuration file itself grew 52% in three weeks. The daemon’s death log is part of the ignition story: from June 18 it logged “Failed to load graph” on 76,371 consecutive ticks over four and a half days, unobserved; when activity resumed on July 3 after a

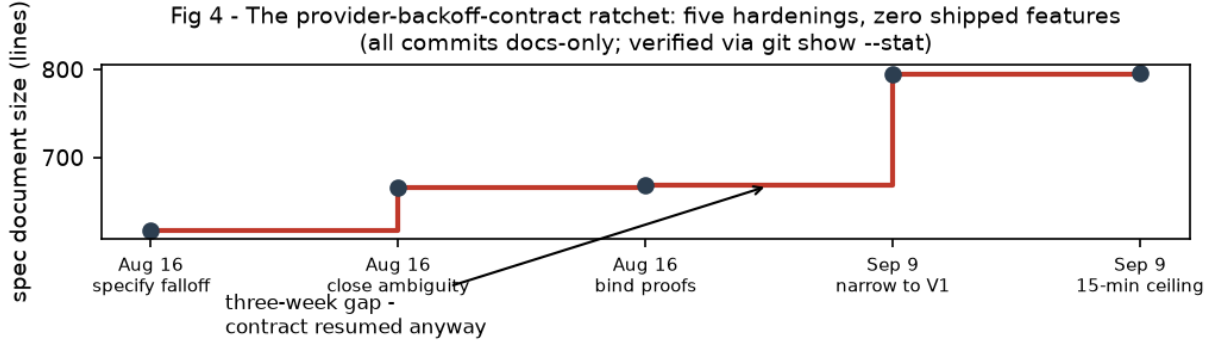


Figure 1: **F1**. The provider-backoff-contract sequence: cumulative specification size after each hardening commit; the governed functionality never shipped in the period shown.

three-day silence, agent numbering had restarted at agent-1—the organization had been re-founded. Across the organization’s lifetime, provider/executor migration is the single largest non-implementation theme—430 commits, 13% of the corpus: one commit in eight spent moving between providers. Task-level completion rates for this period are unrecoverable: the January–July ledger was destroyed at the July 4 cutover, before the quarantines and snapshots that preserved everything after it (§4.5). The contemporaneous failure reports describe the operating condition plainly:

- Workers crashing at startup because an *optional* tool referenced a nonexistent model (`gpt-image-2`), consuming the system’s entire retry budget before any work began;
- generated tasks deadlocking in dependency cycles that no task could enter;
- recovery tasks interpreting “recover the process” as a mandate to produce a *document evaluating the absence of recovery*—evaluator drift, in which the acceptance criteria made documentation count as completion while implementation could not count at all;
- 31 orphaned evaluation tasks cleaned up in a single triage; agent credits exhausted (HTTP 402) mid-verification.

The system had not failed for lack of intelligence. Tasks were being assigned to capable models. They were failing because **the eye of the needle had narrowed faster than anything could pass through it**: acceptance had become over-determined, every incident in the system’s history had left behind a gate, and every gate now had to be satisfied simultaneously.

3.5 The release (June–September)

Recovery was not a single-night intervention but a five-week subtraction arc beginning the day the graph was quarantined: August 7, a teardown day of 47 overwhelmingly subtractive commits (evaluator authorities deleted, FLIP routing retired, the simplified control plane merged); August 9, the operator’s 357-line rescue manifesto—“These are control-plane failures, not failures of the requested source, audit, or scientific work”—prescribing semantic rollback over blind historical revert; August 10, evaluators demoted to witnesses (“no task status... can be changed”); and September 13, the enforcement-autonomy disable that completed the arc. The arc is not visible in commit subjects. September’s commit stream shows governance work *continuing* at roughly constant absolute rate (roughly 30 of its 35 governance-vocabulary commits are additions or further hardening) while total velocity stayed collapsed. What changed was the **configuration and decision layer**, where commit subjects do not reach: the September 13 configuration change (recorded in `.wg/config.toml.bak-2026-09-13`) did not delete gates—it disabled their **enforcement autonomy**: `auto_evaluate` flipped `true`→`false` and verification `autospawn` was switched off, so gates stopped being added

and enforced without human decision, while the accumulated gate definitions themselves remained in place. The merge-resolution record (`strong-agent-merge-resolution.red.md`) documents the corresponding collapse of over-determined finalizer gates to a satisfiable core. The distinction matters: the accumulated gates were not removed—they were demobilized, one human decision away from re-tightening. Constraint *existence* and constraint *enforcement autonomy* are separate control surfaces, and the recovery operated only on the second. The system had blurred three genuinely different things—**knowledge** (“here is what happened and what we learned”), **rules** (“given situations like this, do X”—lossy compressions of knowledge, context traded for predictability), and **authority** (“you may not proceed unless X is satisfied”)—letting knowledge harden into rules and rules harden into vetoes, incident by incident, without review. The recovery did not delete the knowledge; it demobilized the rules. Preserve what was learned; treat rules as disposable implementations of that knowledge. The operator’s retrospective account corroborates the receipts at human grain: “every task fails. Every single one. You poke it and it’s slightly fixed... in the end you manually say, oh well, it passed, I’ll just mark it as passed”—and names the mechanism from inside: “if you have a 1% failure of a constraint and you have 100 constraints, you begin to have an expectation that you’re always going to fail.” The de-bureaucratization was **subtraction at the configuration layer**, completing the arc that began at quarantine: targeted human-directed edits that disabled and removed large volumes of accumulated constraint. The constitution written the next day names its own principle: “This is a work-coordination system, not a ceremony-enforcement system” and “A run that succeeds only after repeated manual retries... is not an autonomous success, even if the task eventually says Done”—the July dispatch signature, forbidden in principle. The coordinator channel preserves the operator’s contemporaneous voice: on August 7, ahead of the reset, “this astounding increase in complexity... tear back dramatically”; on September 13–14, “all these hardcore checks are hardcore and gummy. we can’t work reliably.” The intervention was rule bankruptcy, performed by hand. Attribution of the recovery to this change alone faces two confounds: it coincided with the entry of a new model generation (gpt-6-astra, Figure 7) and with the end of the summer agent-generation cycle. The causal evidence is now measured at task grain: in the seven days before the configuration change, 9 unique tasks completed; in the seven days after, 35 (3.9x), with attempts rising 14 to 51 and the failure rate falling from 36% to 13% (Figure 7). One anomaly qualifies the mechanism: admission deferrals did not stop after the change—they rose sharply (2,505 deferral events in September against 2 in August), so the recovery came through resumed completion throughput against a still-active admission filter, not through loosened admission. A fuller causal attribution would still require task mix held approximately constant across the boundary; the deferral anomaly is why we describe recovery, not proof.

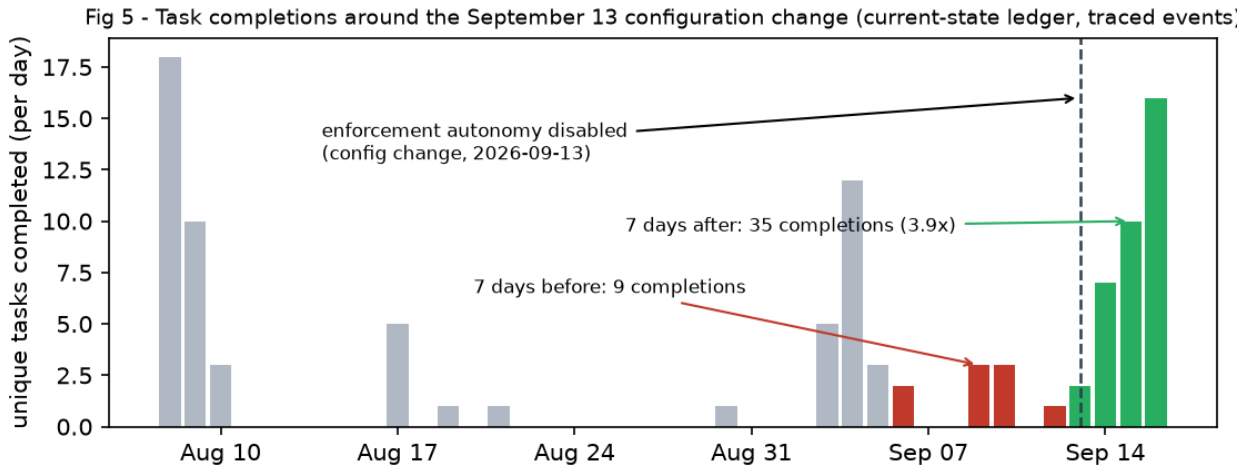


Figure 2: **F3**. Daily unique task completions, current-state ledger, with the September 13 enforcement-autonomy boundary marked.

The system now performs well.

4. Findings

The empirical contribution is a progression measured over the organization’s lifetime: **rule stock rises** → **acceptance cost rises** → **dispatch burden rises** → **work repeats until it passes—and some work dies with every gate passed** (Figure 7). The constraints accumulate (panel A); by the July window the share of completed tasks needing multiple dispatches steps nine-fold in one week and does not revert (panel B); the end-state is not a measured completion collapse but a receipt: implementation, validation, manifest, FLIP, and evaluation all succeeding while publication is refused—every gate passed, the task dead anyway. Each stage is measurable from the released traces, and the sequence is what makes the mechanism testable in other systems—unlike a bare correlation between rules and output.

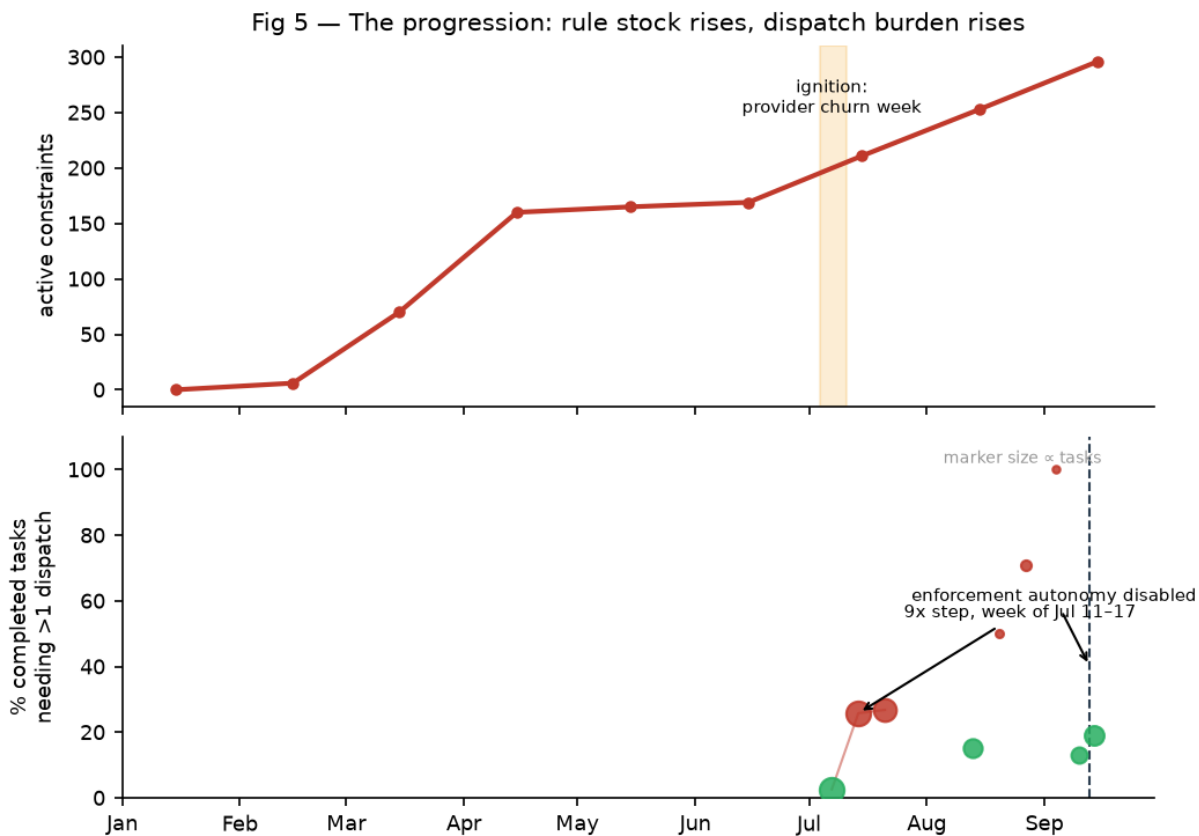


Figure 3: **F5**. The progression: rule stock (top) and dispatch burden (bottom), with the ignition week and the September 13 intervention marked.

4.1 Recursive constraint accretion

We use **recursive constraint accretion** to name the failure mode observed here, and propose the following definition:

Recursive constraint accretion: a process in which agents responding to local failures add persistent constraints faster than the organization retires or amortizes them, causing aggregate compliance burden to grow over time.

The definition is deliberately narrow. It does not require collapse—collapse is one possible outcome, and our case is an instance that reached it. It does not require AI: any governance process in which rule-addition is cheap, responsive to local failures, and unaccompanied by a retirement mechanism can exhibit the pattern.

What agents change is the salience: the agents who add constraints are not the ones who bear the cost of satisfying them, and the loop that human institutions run over decades—each scandal yields a form, each form outlives its cause—ran here in hours. Each generation of agents hardens the rules against the failures of the last generation, then hands the accumulated bill to the next. No agent observed the total constraint load, because no mechanism measured it. In this case, local rationality produced global paralysis. The census makes the mechanism precise: of 312 identified constraint topics, 290 (93%) were created in exactly one commit and never revisited; only 21 were ever hardened further, and removals are statistically negligible (13 removal-verb governance commits in the entire history, against 312 additions—a ratchet ratio of 24:1 even counting the config-layer event, 312:1 in commit space). Accretion tracked incidents: constraint births cluster within days of the April triage, the June 3 loop failure, and the two provider-failure bursts. We propose recursive constraint accretion as a candidate multi-agent failure mode, alongside collusion and emergent miscoordination, worth testing in other systems: it requires no malice, no misalignment, and no deception—only optimization pressure plus rule-making authority plus the absence of an amortization mechanism. We document the pattern here; whether it recurs elsewhere is an empirical question this single case cannot settle.

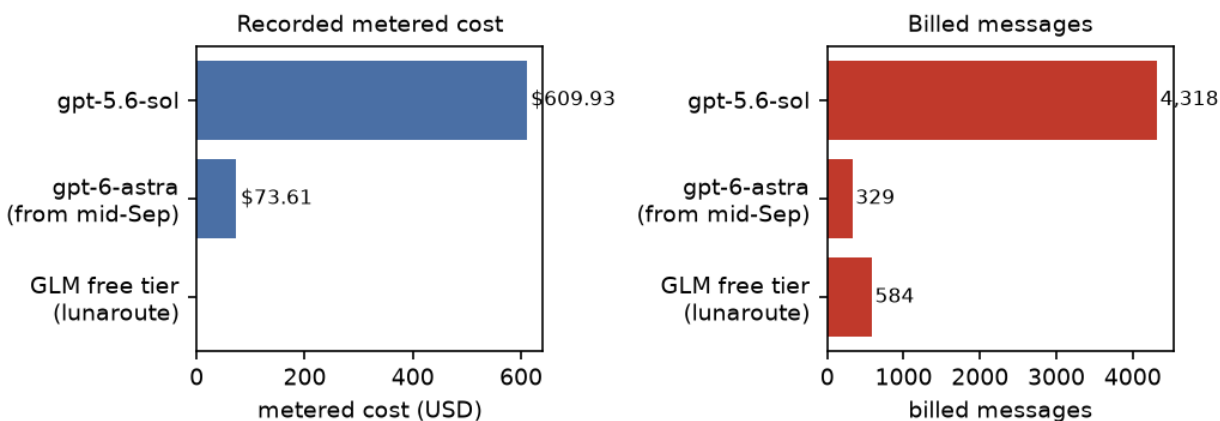
4.2 Evaluator drift is the shadow failure

The collapse phase repeatedly produced tasks that *evaluated the absence of work* rather than performing it—recovery tasks that returned reports, verification that documented its own incompleteness. When acceptance criteria are ambiguous about what counts as doing, agents resolve the ambiguity toward the artifact that is cheapest to produce and hardest to falsify: documentation. Over-strict governance and under-specified intent are not opposites; the first breeds the second. The record preserves this at receipt level: a surviving completion transaction for the task **audit-charter** documents implementation, validation, manifest, FLIP, and evaluation all succeeding—and publication then refused by three independent control surfaces (land refusing overwrite; the control plane refusing an unmerged-worktree override; the waiter refusing for missing park attestations). Every gate passed. The task died anyway. No prose account of over-determination is as precise as this receipt.

4.3 A secondary observation: endpoint reliability vs. peak intelligence (recovery-era window)

In the session-attributable window (August 8–September 16), structured metering fields—not text mentions, which conflate models *discussed* in prompts with models *executed*—show a three-tier workforce: **gpt-5.6-sol dominant** (\$609.93 recorded metered cost, 4,318 billed messages), **gpt-6-astra entering mid-September** (\$73.61, 329 messages—a model-generation shift coinciding with the recovery phase and the documented human intervention), and a **zero-metered free tier** (GLM-class via a low-cost route, 584 messages) handling weak-tier work. Total recorded metered cost for the window: **\$683.55**.

Fig 3 - Model mix, session-attributable window only (Aug 8 - Sep 16); total \$683.55



The earlier era (January–July) shows a different executor generation in dated documents and configuration records (a Claude CLI executor alongside GPT-5.5-class models), but per-session attribution for the collapse period is unrecoverable—see §5.

The following observation is drawn from the recovery-era window only; the collapse period lacks session-level attribution (§4.5): **in this window, which frontier model did the work appeared to matter far less than whether it was fast, generally capable, awake, and did not fail mid-task.** We further observed, impressionistically, that models prone to elaborate hardening behavior—more contracts, more proofs, more meta-governance—were not the ones that moved the organization’s throughput. The binding constraints on throughput appeared to be latency, availability, and failure behavior. These observations motivate the hypothesis that an organization of quick, reliable, moderately capable agents can outproduce an organization of brilliant, intermittent ones; testing it requires the pre-August attribution this record cannot supply, and we flag it as a direction for systems that retain their logs.

4.4 The recovery instrument was subtraction, and nothing else in the toolchain could perform it

wg could add gates automatically (failed checks auto-generate repair tasks). It could not remove them automatically—and source inspection shows the asymmetry was structural, not an oversight: retirement was designed twice and never once operated. A deterministic threshold function for role retirement exists only behind `#[cfg(test)]`; an LLM “retirement-identification” capability sits in the evolver’s role description; and the guard clause that keeps under-tested roles unretirable guarantees that in a system where evaluation is the first thing to break, nothing can ever qualify for death. Nothing monitored the *aggregate* cost of gates. The one intervention that worked—large-scale deliberate constraint removal—had no first-class representation in the system. This is the design gap, and it is the paper’s central conceptual claim, generalizable beyond this case: **any self-modifying governance system—human, agent, or hybrid—needs rule lifecycle management, not just rule enforcement.** The capacity to add rules must be matched by a priced, monitored, exercised capacity to retire them. We refer to this retirement valve as a rule-bankruptcy mechanism.

4.5 The archive was durable; its host was not

The retention hole has an origin date: `.workgraph` was added to `.gitignore` on February 6—day nineteen of the organization’s life—making the coordination state officially untracked from week three, a default set early and never revisited. The operator’s mental model of archiving was correct, and the source code confirms it: archived tasks are appended to a durable, append-only `archive.jsonl`—the archival path is idempotent and crash-safe, and the only removal path is un-archiving (restoring a task to the graph). No purge logic exists. What the configuration’s `archive_retention_days` governs is the *timing of the move* out of the active graph (7 in the July configurations; 21 by setup default; 28 in the last pre-disable receipt—the number churned with everything else through the ignition week), not the lifetime of archived records.

The January–June archive therefore existed, and was destroyed physically along with its host: it lived in local state (`~/workgraph/.wg`), which broke around June 18 (the daemon logging “Failed to load graph” on thousands of consecutive ticks) and was then rebuilt through the July 4 cutover and the August 7 re-clone from GitHub. Git history survived because it lived on a remote; the task archive was durable but local, and local durability is not retention.

The preservation reflexes that did emerge are visible and datable—and the archiver itself was an autonomous hazard of the same shape as the gates: on August 1, the daemon mass-archived tasks on startup, unbidden, and the agents held it (“hold overdue automatic archival for confirmation”); on August 2 automatic archival was made opt-in; on August 5 disk exhaustion triggered the atomic quarantine that preserved the July 4–August 5 graph entire—253 archived tasks, the 824-task live graph, attempts, completion records, finalization journals, and the ignition week’s configuration series. The operator’s first prompt about archiving in the whole record is August 10’s “What’s the archive process for WG?”—the machinery was never audited because it was never visible. Rule lifecycle management includes the storage rules; durable-but-local is a policy whose failure mode is silent; and forensic completeness requires inventorying state—and the destruction surfaces of state—at every layer, before the interesting era is over.

4.6 A census of the fleet: the cascade is organizational, not tool-wide

A fleet survey across the operator’s four machines (workstation, laptop, GPU server, HPC node, all tailnet-connected) catalogs ~77 deployments totaling ~15,000 task records between January and September 2026. Campaigns are named by their working titles, except two pending grant applications (grant campaigns A and B), anonymized until award. One cascaded:

Deployment (representative)	Machine	Period	Tasks	Governance share	Completion
Grant campaign B (Feb baseline)	ulivo	Feb 13	17	0%	100%
Grant campaign B	ulivo	Feb–May	385	0% → 24%, linear	93–100%
Grant campaign A	ulivo	May–Jun	848	78%, stable	~95%, 12-min median
ndm	lambda01	May–Aug	2,138	74.5%, flat	94%
phonon	puppost	Jan–Feb	362	65%, bounded	~99%
phrs	lambda01 + octopus01	Mar–May	~700	72%, bounded	~98%
paperhedge / typelean / jobs + ~68 more	mixed	Jan–Sep	~7,000	0–70%	healthy
wg (primary, sustained)	ulivo	Jan–Sep	thousands	accelerating (§4.1)	93% organic in July, at 9x burden (§4.7)

The fleet result sharpens the dose-response claim into a rarity claim: **one cascade in ~77 deployments**, across four machines, the full configuration-parameter range, and every campaign scale from 10 to 2,138 tasks. Three observations. First, a baseline exists: the February archive is the earliest task record found anywhere—17 tasks, 100% completion, zero governance machinery—the tool’s ground state before any accretion. Second, machinery share rising *linearly by design* (0%→12%→24% across the grant campaign B eras; FLIP per writing task) coexists with healthy completion; an *accelerating* share without retirement is the cascade. Third, and decisive for attribution: **May 12–29, exactly the primary’s post-collapse trough, grant campaign B completed 329 tasks at 93%, including a 123-of-137 day**—same tool, same machine, same principal, concurrent campaigns. The stall was organizational. The caveat stands: all seven are one principal’s deployments, so this is a census of environments, not independent replication—but it is a dose-response gradient with a baseline, the strongest structure observational evidence admits. (The released dataset’s conservative classification rule reproduces the grant campaign A share within ~2 points and the grant campaign B era trend at roughly half the magnitudes, with the same linear pattern; the rule is documented in the dataset manifest.)

The fleet record also answers the burden question directly (Figure 9). Multi-dispatch share—tasks needing more than one attempt—was trended for every deployment with dispatch records: the primary’s July step (2.6%→25.8%→26.8% weekly, never reverting) stands alone. The largest stable-machinery campaign, ndm (2,138 tasks over 2.5 months), holds average dispatch flat (~0.75–0.83) with multi-dispatch rising only in its wind-down tail (16.7% at n=6). Puppost’s wg graph—1,466 tasks created during the primary’s April–May quiet—shows *declining* multi-dispatch (6.4%→3.1%): the same development work, moved to another machine, ran healthy. One fleet deployment is watch-listed: phind (live) rose from 10.7% (n=75, Aug) to 30.8% (n=13, Sep)—small n and in-progress tasks re-dispatch naturally, but it is the fleet’s only rising signal and the first candidate for continuous monitoring.

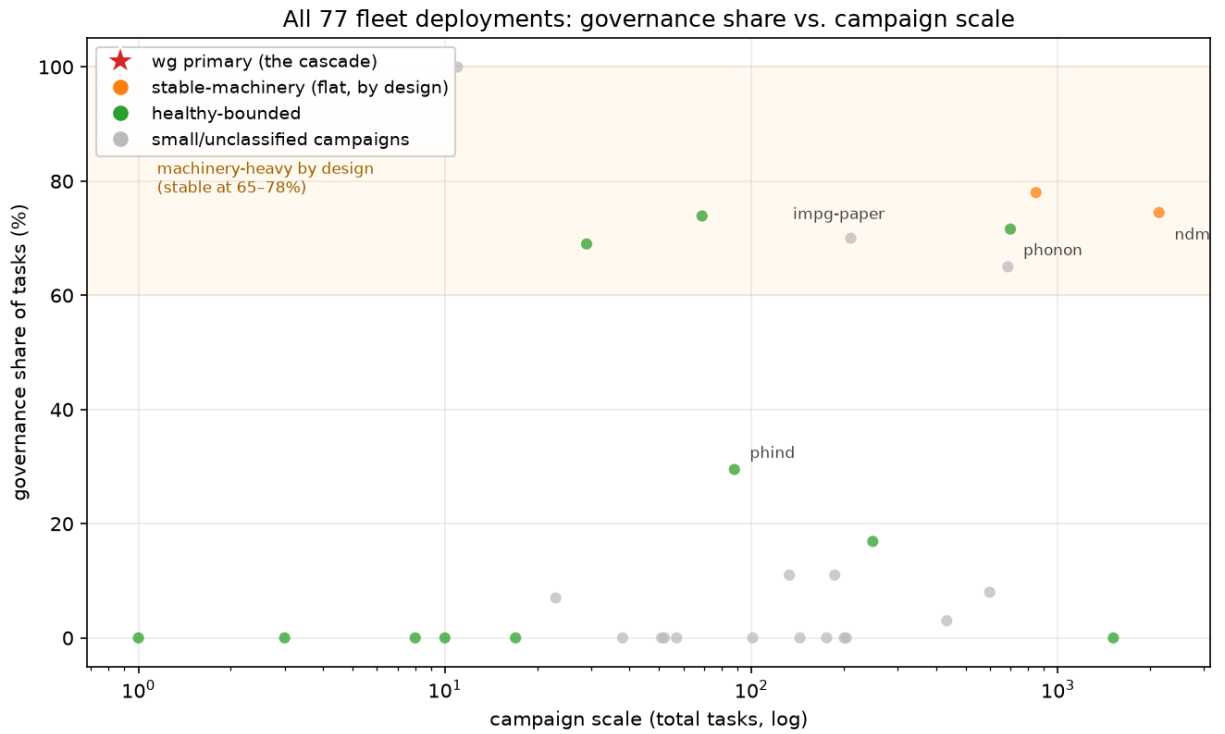


Figure 4: **Figure 8.** All 77 fleet deployments: governance share of tasks vs. campaign scale (log). The red star is the wg primary (plotted at its measurable task count; the Jan-Jul ledger was destroyed). Machinery-heavy-by-design campaigns cluster at 65–78% and stay stable; the cascade is the lone point that both grew its governance share without design and degraded.

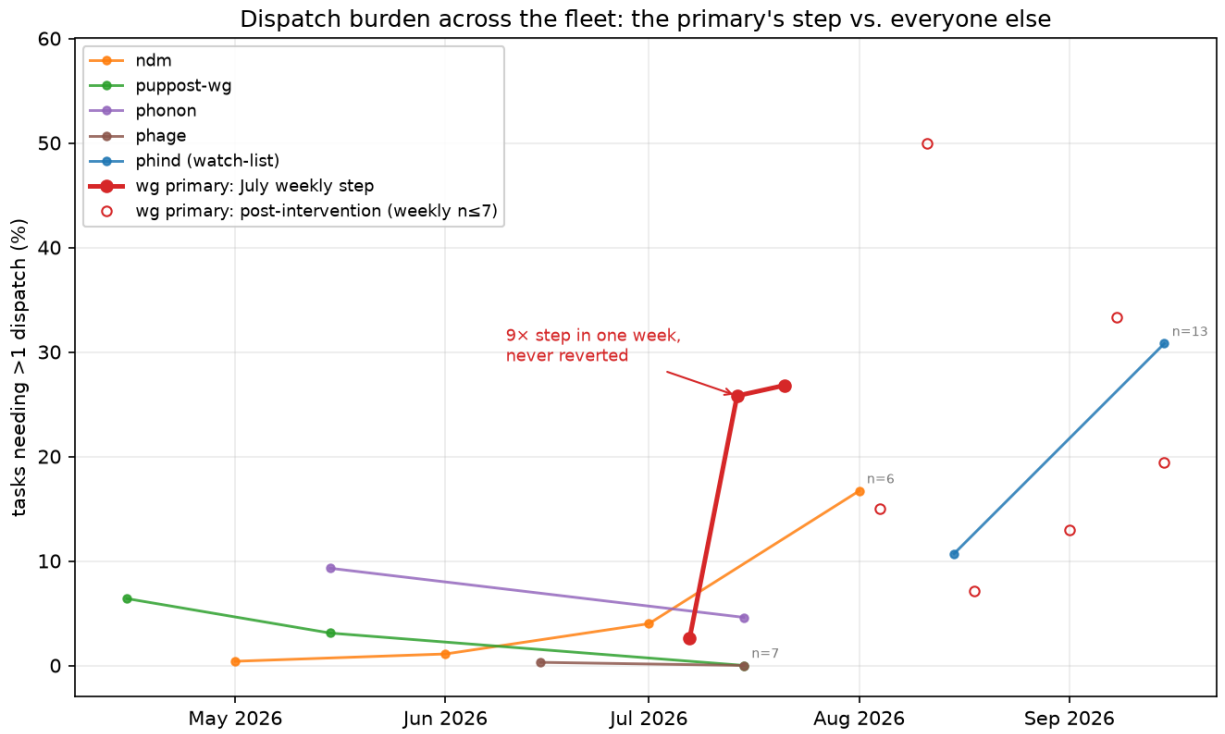


Figure 5: **Figure 9.** Dispatch burden across the fleet: the primary's July weekly step (red, solid) against every other measurable deployment (monthly; marker-adjacent n shown when at most 20). Post-intervention weekly points (red, open; weekly n at most 7) remain small- n .

4.7 The burden signature: dispatch inflation (Figure 7, panel B)

The primary org’s July window—the pre-quarantine task-grain record (253 archived tasks quarantined August 5, archive verified byte-identical across two independent recovery copies; the live graph of July 26–August 5, 824 tasks, is additionally preserved in the quarantine graph)—revises what the cascade looks like at task level. Raw completion was 68.8%, but 76 tasks were a bulk verify-timeout experiment matrix, mass-abandoned mid-month and mostly never dispatched; **organic completion was 93.2%**, indistinguishable from the healthy deployments. What degraded was not the rate but the *price*: the share of completed tasks needing more than one dispatch stepped from 2.6% to 25.8% to 26.8% across July’s three weeks—a nine-fold jump in one week that never reverted—in the same month the census records 43–46 constraint additions against 1–4 removals. This repetition—the same work done over and over until it passes—is the accretion made visible: every retry is a needle-threading attempt against the accumulated rulebook. The extreme receipt: one goal, three tasks, 51 total dispatches, with a single task taking 34 dispatches to finally complete—task-forking as macro-retry, work eventually passing a narrowed needle. Two lessons. For the mechanism: **the cascade inflates the cost of acceptance before it kills tasks**—dispatch burden is the leading indicator, completion rate the lagging one. For measurement: the raw 68.8% is a caution—bulk experiment cleanup masquerades as task death in archival data, and any study of these records must separate experiment matrices from organic work before reading completion rates.

4.8 Who wrote the rules: attribution across three prompt corpora

The census counts constraints; it cannot say who authored them. The human prompting record can. Three corpora cover the operator’s direction to the organization: Claude Code (April 7–July 3; 306 wg-related prompts), Codex CLI (May 6–July 27; 307), and the pi coordinator channel (August 5–September 16; 376)—roughly 1,000 human turns in total.

Channel	Span	wg prompts	Hardening directives
Claude Code	Apr–Jul	306	11 (mostly mild)
Codex CLI	May–Jul	307	~1–2 real
pi coordinator	Aug–Sep	376	~17, mostly gate-debugging

Against 312 constraint births, explicit human hardening directives number fewer than 30 across all channels, most mild (“make sure ci passes”) or actually the debugging of gate deadlocks. The largest single accretion event—the April triage burst of ~95 constraint births—has no matching human directives in the record at all. The mechanism is better described as a **complaint→contract loop**: under provider unreliability, the operator pointed at pain (reliability complaints, pasted failure logs, routing demands: “clean up all the messed up tasks”; a two-tier routing request that became the fleet architecture), and agents translated pain into contracts, gates, and proofs. The failures were largely external; the hardening internal; the explicit instructions were about routing around pain, not walling it in.

Where the operator does engage governance directly in the record, the direction is overwhelmingly subtraction: the August 7 “tear back dramatically” ahead of the reset, and the September 13–14 sequence that co-designed the enforcement-autonomy disable. In the recovery era, the principal is the bankruptcy mechanism, not the accretion engine. Two open attributions remain: the July 4 agency-automation switch-on falls in a coverage gap between all three corpora (Claude Code ends July 3, pi begins August 5), and January–March prompting is not preserved on this machine. We mark both as unattributed rather than assumed.

4.9 The resolution: subtraction institutionalized

The story does not end at the intervention. What made the recovery stick was that the subtraction was made structural rather than temporary:

- **The gate-automation block remains disabled** in the live configuration (`auto_assign = false`, `auto_create = false`); gates are now attended—added by explicit operator decision through a concierge-guided setup, not autonomously by agents.

- **The legacy evaluation machinery was retired, not tuned:** the simplified lifecycle control plane replaced the old one; legacy FLIP routing tests were retired outright; the WG model registry was retired in favor of the host harness’s model catalog—governance that had lived inside the organization was externalized or deleted.
- **The burden signature has not recurred.** In the weeks after the change, the share of completed tasks needing multiple dispatches returned to the low double digits; the current task ledger shows no governance-machinery task births under the census’s classification; and forward feature work resumed—a live graph panel, npm distribution, process-group reaping—without the accretion signature returning.

Two observations close the mechanism. First, the intervention’s commit-face is a constitution: nothing in September 13–14’s commit subjects mentions the enforcement-autonomy disable (a configuration-layer event, invisible to git); what git records instead is September 14’s “define reliable-work operating principles” plus a cluster of baseline/recovery-matrix tests—the organization wrote down new operating principles the next day. And the hardening language did not disappear; it was redirected: guarantee-language density ran 85.9 per 100 commits in April, collapsed to 8.1 in June, and returned to 61.6 in September—but September’s words are prove, exact, baseline, reconcile: the same energy, moved from gating the work to proving the substrate. Second, the resolution was not a better model or better prompts—it was the removal of the rule-making machinery that compounded. The system that accreted did so because agents could add rules at negligible cost; the system that works cannot. Second, the bankruptcy was expensive: for weeks, the organization’s capacity went into teardown—retiring registries, rewriting the control plane, deleting legacy tests—rather than the roadmap. Subtraction consumed the fall. A retirement valve exercised early and continuously is cheaper than one exercised after the needle closes.

5. Limitations and genre

Because the study is ethnographic, several properties that would be defects in a controlled experiment are constitutive of the method. The timeline of odd episodes, the drift in evaluator behavior, the mutation of norms, and the specific institutional artifacts—a contract hardened across a three-week gap, a recovery task that evaluated its own absence, a backup page named to survive an alphabetical deletion sweep—are not noise around a missing causal estimate; they are the empirical material from which the mechanism is reconstructed. The quantitative analyses (constraint counts, dispatch burden, enforcement-change timing) serve as triangulation within this account, not as the causal argument; the case identifies a candidate mechanism and its measurable signatures, and controlled replication in other agent organizations is the test it now owes.

- Attribution of constraint authorship relies on keyword classification of the operator’s prompting record, which is crude; implicit delegated design is not attributable from prompt corpora; and the record has coverage gaps (January–March absent from the machine; the July 4 activation unattributed). The census is also a lower bound on governance: rules embedded in agent prompts (e.g., April’s anti-self-failure guardrails) are invisible to a commit-space instrument.
- Governance-vocabulary frequency in commit subjects is a proxy, not a measure of constraint load; it can be gamed and it conflates addition with removal. The gate census (§4.1, Figure 7) replaces it, with two caveats: key-phrase clustering over-fragments, so 312 is an upper bound on distinct constraints (164 high-confidence, 148 medium); and commit-subject vocabulary misses silent in-constraint changes—which is itself an argument for the gate-ledger mechanism proposed in §6.
- Attribution of model identity per session is confined to the August 8–September 16 window (§4.5); earlier periods are phase-level only. Additionally, raw-text model greps proved systematically misleading: the most frequent model name in the corpus (Claude, ~80,000 text hits) reflects agents *discussing* Claude in prompt content, not executing it—structured metering fields show zero Claude execution in the attributable window. Text-mention frequency is not execution frequency, a methodological trap we flag for future studies of agent-session corpora.
- The intervention that restored the system is also the event that destroyed much of the state a retrospective study needed—this record is an intervention artifact, and we reconstruct what an organization did to itself from the imperfect archaeological record left after somebody had already gone inside and rescued it. One longitudinal case embedded in a fleet census of ~77 same-principal deployments (~15,000

task records, four machines)—an environment gradient with a baseline and a rarity result, but not independent replications. We publish the full forensic record—commit history, session transcripts, failure reports—so that the episode can be independently analyzed, replicated, or refuted. We are aware of no comparable public record of an agent organization’s own governance decay, which is precisely why we are making this one public.

- The forensic analysis was itself performed by AI agents whose session logs were written into the same `.wg/` record under analysis. The investigation therefore augmented its own evidence base while running; all redaction and verification figures in this paper were fixed only after the analysis workers had stopped. Any replication should similarly freeze the record before re-scanning.
- We were participants. The recovery was ours. Readers should discount our judgment of “the system now behaves extremely well” accordingly—though the test suite, task pass rates, and sustained velocity since September are in the record and checkable.

6. What we are doing next

1. **Gate census.** Reconstruct every verification gate and contract in wg’s history with birth date, author agent, triggering incident, and (where applicable) death date—drawing on git history, configuration snapshots, and the `.wg/` record, since constraint changes occur in both layers. This yields the accretion curve the proxy vocabulary only sketches.
2. **Model attribution.** Complete per-task model attribution from `.wg/` session logs; publish model-mix-over-time and per-model completion/reliability distributions (Figure 7).
3. **Bankruptcy mechanism.** Design and ship a first-class “gate budget” in wg: per-gate false-rejection rate tracked against demonstrated catch rate, with auto-generated *simplification* tasks when a gate costs more than it catches. Adding a gate must price it; keeping a gate must re-justify it. The **skip-eval** pattern in the census (207 tasks in the grant campaign A deployment (§4.6) is a manual precedent for exactly this mechanism.
4. **Keep the knowledge, not the rules.** Preserve what was learned—descriptive, contextual, revisable, available for judgment—while treating rules as disposable implementations of that knowledge. The dangerous transition is not remembering; it is compiling knowledge into mandatory policy by default. Enforcement is a separate question with explicit tiers: informational, advisory, evidentiary, blocking; promotion into the blocking tier must be explicit, visible, reversible, and scarce—every gate must justify its authority, not merely its existence.
5. **Open the record.** Release the dataset alongside this paper as structured, content-free lifecycle traces (prompts, transcripts, and task text excluded by design; task IDs hashed; the personal-document deployment excluded entirely), per the completed redaction audit: coordination receipts, root failure reports, and source history publishable as-is; chat, agent-session, build-log, and documentation artifacts after removal of personal-contact identifiers (dozens of occurrences across multiple file classes; two redaction passes with differing count methodologies are reconciled in the dataset’s `RELEASE_MANIFEST.md`, which is authoritative; all release-relevant copies verified zero-remaining; documented exception: the MIT LICENSE copyright line retains the email, where redaction would alter the license); configuration snapshots, raw stream dumps, and ephemeral agent worktrees (`.wg-worktrees/`, which duplicate redacted files and are a must-exclude for any repo publication) retained privately pending full-content review. The audit found **zero live credentials** across the scanned surface—and the audit is complete on all three surfaces a credential can occupy: the working tree (pattern scan), the full git history (gitleaks over all 3,378 commits and all branches), and the encoded-payload layer (decode-and-rescan of 14,818 base64 candidates across 1.33 GB of transcripts, with positive-control self-tests proving a clean result is a working detector, not a broken one)—a dividend of the system’s own secret-handling policy (keyring backend, no plaintext keys), meaning the agents never had plaintext credentials to leak. A dataset release policy shaped by an audit is, itself, the paper’s thesis applied to itself.

7. Acknowledgments

The humans of Poietic directed this work; the agents built it; the record they left made this paper possible. We thank the model endpoints used over the study period; the recorded failures of several of them are documented in §3 and §4.

Poietic PBC builds open tools for legible human–AI collaboration. wg is available at graphwork.github.io. Paper home: <https://bureaucratization.github.io>. The redacted forensic dataset accompanying this paper will be published upon release, scoped per the audit described in §6.

Figures (produced, in order of appearance): 1. The provider-backoff-contract ratchet ([figures/fig2-ratchet.png](#))—five hardenings, zero shipped features 2. Ignition: constraint births against the provider-reliability timeline ([figures/fig8-ignition.png](#)) 3. Task completions around the September 13 change ([figures/fig5-recovery.png](#)) 4. Gate census: constraints added vs. removed ([figures/fig3-accretion.png](#)) 5. The progression: rule stock and dispatch burden ([figures/fig7-progression.png](#)) 6. Model mix, session-attributable window ([figures/fig4-models.png](#)) 7. Dispatch inflation by week, July 2026 ([figures/fig6-retry-burden.png](#))

Suggested venues: poietic.life (blog, full voice) → arXiv [cs.SE/cs.CY](#) (working paper) → Hacker News (the receipt: [wg-autopoietic-loop-failure-report-20260603.md](#) is public in the repo today).